

AI 使用说明情况报告

一、使用的 AI 模型

使用的 AI 大模型名称	在实验过程中的作用
DeepSeek	它在我遇到未知报错和需要快速搭建模拟环境时，扮演了“高级搜索引擎”和“代码结对审查员”的角色
WorkBuddy	通过语言描述步骤自主生成相应的流程图和可编辑文件，大大节省了实验时间成本
CherryStdio	辅助阅读和理解文献，并给出相应详细的解释

二、使用的 AI 提示词（Prompt）示例

我尽量使用结构化的提示词来寻求帮助，核心举例如下：

- “我在 WSL2 安装时遇到 0x800701bc 错误，如何在不重装系统的前提下解决？”
- “SGLang 启动时报错 ModuleNotFoundError: No module named 'vllm'，我是纯 CPU 环境，请问如何绕开或者替代？”
- “云服务器上运行 SGLang 报错 fatal error: cannot execute 'cclplus'，该如何解决？”
- “请帮我写一段 Python 脚本框架，要求基于指数分布模拟 Poisson 到达过程，并记录 15 个请求的 Latency 和 TTFT。”

三、AI 如何帮助我学习，以及我在哪些方面使用了 AI

1. 系统级排错与底层依赖定位

我本身对 Linux 和 CUDA 环境并不熟悉。昨晚在 WSL 和云端环境折腾时，遭遇了假安装包、Python 3.14 缺少 Rust 编译器等一连串问题。AI 帮我迅速定位了报错日志的核心（比如告诉我 cclplus 需要安装 build-essential，TorchAudio 和 PyTorch 的 CUDA 版本不一致需要卸载重装），极大地节省了我盲目试错的时间。

2. 概念理解与核心问题解答

在“重点回答”部分，关于 RadixAttention 和 PagedAttention 的区别，我查阅了 SGLang 和 vllm 的论文并借助 CheeryStudio 理解。在把论文中晦涩的学术语言转化为工程直觉（比如“内存管理层”和“前缀语义层”）的过程中，AI 帮我把复杂概念结构化、生活化，让我理解得更透彻。

3. 代码辅助生成与环境调试

在构造 Mooncake workload 阶段，由于本地资源受限，我向 AI 请教了模拟脚本的逻辑。AI 给了我基础的 Python 代码框架，我在本地亲自完成了环境的激活、语法调试和几次路径报错修复，最终成功跑出包含 tokens、status、latency 和 TTFT 的完整日志。

四、运用 AI 时是否被误导（独立思考与反思）

有被误导过。最初，我在本地 AMD 显卡的 Ubuntu 环境下报错缺少 vllm 时，AI 曾建议我直接安装 vllm 模块。但我在尝试时发现，这会强迫系统下载几个 G 的 CUDA 依赖并在编译时失败。我意识到 AI 没有完全考虑我“无 N 卡”的底层硬件限制。

于是我及时止损，拒绝了 AI 的初始建议，果断转战云端 GPU 方案，从而挽救了整个进度。这个经历让我明白：AI 给出的方案往往具有普适性，但在实际的系统环境中，必须结合自身的硬件条件、性能代价和时间成本去独立决

策。

五、总结

AI 极大地加速了我从“未知”到“跑通”的学习曲线，帮我梳理了知识盲区。但我清楚地知道，AI 可以给出代码框架和排错方向，但最终执行命令、判断方案优劣还需要靠我自己进行判断决策。未来在实验室的科研中，我会继续保持“AI 辅助 + 独立思考 + 手动验证”的工作模式。